ZAVIONIX / ETHOS

Autostart
User Manual

A turbine start/cooling controller with touch and RC-switch controls. It publishes starter and igniter Lua sources for use in ETHOS mixers.

Widget version 3.1.12 | English | 15 October 2026

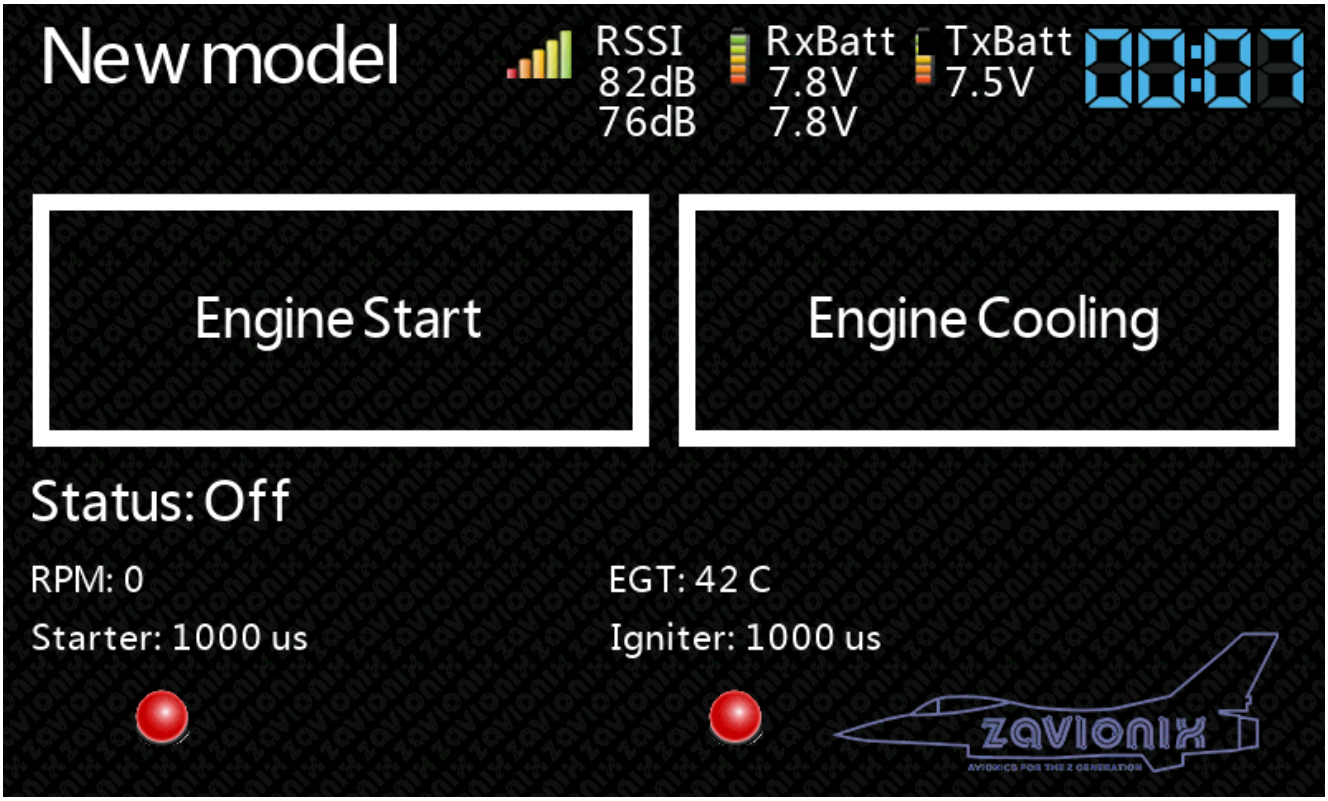


Figure 1. Actual ETHOS simulator rendering with simulated sensor data. The example is not a flight record or a recommended operating setup.

Read this guide in order for first installation. Use the settings and sensor reference when adapting an existing model.

1. Before you begin

A turbine start/cooling controller with touch and RC-switch controls. It publishes starter and igniter Lua sources for use in ETHOS mixers.

You need a compatible color-screen ETHOS transmitter or simulator, the complete widget folder, and the relevant telemetry sources already available in the model. These captures use the VS Code-cached X20S ETHOS 26.1.3 simulator engine at 800 x 480. Other screens may arrange or scale the content differently.

This manual covers installation, configuration, every captured widget-specific form field, normal use and troubleshooting. Menu labels are written as they appear in this version. Numeric examples are demonstration settings, not model-specific recommendations.

Guide contents

2. Install and add the widget

3. Configure telemetry and read the display

4. Operating guide

5. Settings and sensor reference

6. Troubleshooting and verification

7. Screenshot provenance and version record

2. Install and add the widget

1. Back up the model and existing widget folder. Download the Autostart repository ZIP from https://github.com/i3dm/Autostart (Code / Download ZIP) or use the appropriate version of the repository.

2. Extract the archive and copy the whole src/Autostart folder into the radio storage /scripts directory, producing /scripts/Autostart/main.lua. Keep the companion Lua files, images and audio in their supplied relative locations. Do not copy only main.lua.

3. Restart ETHOS or reload Lua through the facilities supported by your firmware. If upgrading, use one complete version of the package rather than mixing old and new module files.

4. Open Configure screens (DISP in the simulator). Add or select a screen and choose a suitable layout; for a full-screen view, long-press its tab and use Change layout. Tap the widget area and select Zavionix Autostart from Widget.

5. Open the widget configuration, set the model-specific parameters, and assign telemetry sources. Use the back arrow to return to the dashboard. Reopen the settings and confirm the assignments persist.

Use only the production widget folder on a real transmitter. Documentation sensor fixtures and simulator radio copies are not part of the installation.

3. Configure telemetry and read the display



Figure 2. Widget configuration in ETHOS. Scroll down to reach the remaining settings and source selectors; the reference lists fields below the visible area as well.

Power the intended telemetry hardware in a controlled setup and discover its sensors in the ETHOS model telemetry page. Confirm each value and unit there before assigning it in the widget. Receiver 1/2 and engine left/right inputs must refer to the intended physical source.

A missing value or a No ... label normally means an unassigned or unavailable source. A plausible but incorrect value often means the wrong source, unit or scale. Inspect the source in ETHOS telemetry first; do not compensate for a bad assignment by changing an unrelated capacity or alert setting.

Where displayed, RSSI and RxBatt come from your selected receiver sources; TxBatt comes from the transmitter. The model name and picture come from ETHOS model settings. A missing model image does not imply a telemetry failure. The example photograph is specific to the test model.

4. Operating guide

Connect the controller to ETHOS mixers

This widget can command starter and igniter outputs. Before connecting their power, test the entire setup with the actuators disconnected and observe the ETHOS channel monitor. The simulator screenshots show software behavior only; they do not verify physical wiring, channel reversal or engine suitability.

Install the complete Autostart package, make its Lua sources available through the model's Lua configuration, and add the widget to a screen. Select Autostart Starter and Autostart Igniter as the appropriate mixer sources for the two intended output channels. Verify those source names in the selector before connecting hardware.

Internally, 1000 us corresponds to the off request (-100%), 1500 us to the midpoint and 2000 us to +100%. Channel weights, offsets, reversals and output limits can change the actual pulse at the receiver. Measure the receiver output and verify both off states, the 1800 us starter example and the 2000 us igniter example.

Assign the correct ECU Type, EGT and RPM sources. Set the start and cooling RC controls and their delays. Check touch commands and switch commands independently. Confirm that canceling a sequence returns both published outputs to off. Arrange fuel/ECU control separately; the published outputs are starter and igniter.

Start sequence and status messages

A start request begins the initial starter phase, then preheating with the igniter on and starter off. The temperature must rise by 5 C from the baseline within Ignition time out; otherwise the controller reports Low Pre heat temperature and stops the sequence.

During acceleration the starter ramps toward its configured value while the igniter remains requested. The logic checks RPM progress; failure to increase sufficiently can report Low starter RPM. At Self sustained RPM, starter and igniter are turned off and the controller verifies continued running.

Verification accepts a further RPM rise of 500 RPM or holding the self-sustained threshold for three seconds. It reports Start successful when the check passes, or RPM did not climb when the expected running condition is lost. These checks are software criteria and must be matched to the real engine.

Activating Start while a start sequence is active cancels it and reports Start canceled. The pictured idle example uses 0 RPM and 42 C. It does not claim that a complete engine start was simulated or validated.

Cooling sequence

A manual cooling request waits for RPM to fall below Cooling Start RPM, then alternates the starter-on and starter-off intervals. The igniter remains off. Cooling ends when EGT falls below 100 C and reports Cooldown successful.

Automatic cooling can begin while idle when EGT is above 100 C and RPM is below the cooling threshold, unless it has been suppressed by cancellation. Activating Cooling during cooling cancels the sequence, suppresses that automatic cooling cycle and reports Cooling canceled.

Do not use a delay, threshold or PWM example from this manual as an engine manufacturer's recommendation. Verify source loss, cancellation, restart and physical output behavior on a controlled bench before enabling powered starter/igniter equipment.

5. Settings and sensor reference

The following labels were read from the running widget configuration. Example numbers describe the documentation setup or initialized fields; they are not guaranteed factory defaults for a previously saved model. Source assignments and switches must be chosen for your model. Decimal values below use the units displayed in the form.

Parameters

Menu field	Choices / example / range	Purpose
Engine On RC Switch	Unassigned; select a switch	Assign the RC control that requests a start sequence. Use a deliberate switch position and verify its inactive state first.
RC Switch Delay (s)	Example 0; range 0 to 10	Delay applied to RC-switch activation, in seconds. Verify its behavior with outputs disconnected.
Touch buttons delay (s)	Example 0; range 0 to 5	Required touch-button delay in seconds before triggering a command.
Starter on PWM (us)	Example 1800; range 1000 to 2000	Requested starter-on PWM equivalent in microseconds. Output scaling and channel endpoints must be checked in the mixer.
Igniter on PWM (us)	Example 2000; range 1000 to 2000	Requested igniter-on PWM equivalent in microseconds. The off equivalent is 1000 us.
Self sustained RPM (K)	Example 10; range 1 to 50	RPM threshold, in thousands, at which the controller turns starter and igniter off and verifies that the engine remains running.
Starter ramp up time (s)	Example 2; range 0.1 to 5	Time in seconds for the starter request to ramp from off to its configured on value.
Initial Starter time (s)	Example 3; range 1 to 10	Duration of the initial starter phase before preheating.
Ignition time out (s)	Example 5; range 1 to 10	Time allowed for the preheat temperature-rise check before the sequence is aborted.
Engine Off switch	Unassigned; select a switch	Switch used to request cooling. The widget publishes starter/igniter outputs; confirm your separate fuel/ECU shutdown arrangement.
Cooling Start RPM	Example 2000; range 1000 to 10000	Maximum RPM for starting the cooling starter pulses.
Cooling starter ON time (s)	Example 3; range 0 to 10	Duration of each starter-on cooling pulse, in seconds.
Cooling starter OFF time (s)	Example 10; range 0 to 60	Pause between cooling starter pulses, in seconds.
ECU Type	Jetcat; Projet; Xicoy V6-V10; Kingtech G2-G5; Jetcentral; Swiwin; Xicoy X; Orbit; AMT; Linton V1/V2/V3; EnJet. Example: Kingtech G2-G5.	Select the actual ECU family before assigning telemetry. This controls status decoding, scaling and fuel calculations.

Source assignments

Source selector	Required value and interpretation
Temp1 Sensor (EGT)	Exhaust gas temperature in degrees Celsius. Use the ECU temperature channel, not its status channel.
RPM Sensor	Engine-speed telemetry. The turbine decoder can apply ECU-specific scaling; compare the displayed RPM with the ECU reading.
RSSI 1 Sensor	Receiver signal-strength source in dB. Keep receiver 1 and receiver 2 distinct; leave an unavailable second link unassigned.
RSSI 2 Sensor	Receiver signal-strength source in dB. Keep receiver 1 and receiver 2 distinct; leave an unavailable second link unassigned.
Rx Batt 1 Sensor	Receiver supply voltage in volts. This is separate from transmitter voltage and, where applicable, the main propulsion battery.
Rx Batt 2 Sensor	Receiver supply voltage in volts. This is separate from transmitter voltage and, where applicable, the main propulsion battery.

6. Troubleshooting and verification

Symptom	Checks
Widget absent from selector	Check /scripts/Autostart/main.lua and all companion files. Restart ETHOS and inspect Lua messages. Make sure the archive did not create an extra nested folder.
Error after update	Restore a complete matching package version. Check available Lua memory and the exact error in the simulator/radio log; retain the model backup.
Missing / No sensor value	Inspect Model / Telemetry, power the sensor, rediscover if necessary, then reselect the correct source in the widget.
Value has wrong magnitude	Check units, decimals, ECU selection and upstream sensor scaling against a known reference. Do not change warning thresholds to conceal a scaling error.
Values do not change	Compare the original telemetry page. A retained or stale last value is not proof of an active link. Confirm fresh telemetry before use.
Text is clipped	Use a larger widget area/full-screen layout. Cells also has Font Size. Check the intended screen size and version.
No audible alert	Verify enable switch, threshold, repeat interval, telemetry validity, audio files and transmitter volume. Headless screenshots cannot verify the speaker.
Settings appear different	Confirm the selected model, widget instance and, for combined widgets, Widget Display. Each can have its own saved settings.

Model acceptance check

1. Reopen the saved model and verify every configured source and unit.

2. Compare the widget with the underlying telemetry at two or more known values. For twin/battery pairs, vary one side at a time.

3. Check resets, timer controls and any alert conditions relevant to your model in a controlled test. Observe the physical transmitter audio where alerts matter.

4. Verify behavior when a source becomes unavailable and after restoring it. Do not infer source-loss behavior from a static screenshot.

5. Record the final capacities, thresholds, scaling and calibration values with the model backup.

7. Screenshot provenance and version record

The screenshots were produced by the genuine ETHOS X20S 26.1.3 engine (a074fdfd) cached by the VS Code ETHOS extension, driven headlessly through the ethos-tools PR #2 runner. This validates Lua execution and rendering in that engine; it is not a claim that the VS Code webview or every supported transmitter was tested.

Documentation-only copies supplied genuine ETHOS sensor objects and assigned them to the widget. The rendering implementation was retained. Most inputs used DIY sensors refreshed with rawValue; Cells used injected S.Port cell frames to provide an actual multi-cell source. Demonstration settings were initialized independently of the copied model. These fixtures were not installed in the production widget repository.

The simulation method was informed by WingFlight's simulator sensor implementation. The example values are synthetic, not flight measurements. Static captures and configuration inspection do not establish calibrated measurement accuracy, full alert-state coverage, audible playback or physical control performance.

Widget: Autostart 3.1.12. Source commit inspected: 9253cdb4ccb2c20a813e1e0e5d39e8368949ab49. Documentation edition: 15 October 2026. Repository: https://github.com/i3dm/Autostart.

Source links

Widget source and updates

ETHOS simulator automation PR

WingFlight simulator sensor approach